

Software Requirements Specification

Team3-Basys-Editor

Version Control

Version	Date	Author	Comment
1.0	23.10.25	Federico Dibenedetto	Initial creation
1.1	30.10.25	Federico Dibenedetto	Changed UC1 based off of weekly findings and updated flowcharts to use mermaid
1.2	11.11.25	Federico Dibenedetto	Changed Use Cases based off of meeting with Mr. Rentschler
1.3	13.11.25	Federico Dibenedetto	Added Glossary, Component overview and little fixes
1.4	16.04.26	Federico Dibenedetto	Changed Use Cases based on Stakeholder meeting and meeting with BaSyx developer
1.5	05.05.26	Federico Dibenedetto	UC 4 and FR feature request adapted
1.6	19.05.26	Federico Dibenedetto	Added links to BaSyx Feature-Issues according to Stakeholder requirements

Table of contents

1. Glossary

2. Scope

3. Introduction

4. Component Overview

5. Use Cases
- 5.1 UC01: Import with plausibility check and MIME type detection

• 5.2 UC02: XML viewer with navigation and display functions

• 5.3 UC03: AAS generator from KBL/VEC

• 5.4 UC04: Automated extraction of specific XML entries from the AAS
6. Customer Requirements

◦ 6.1 Functional Requirements

▪ 6.1.1 FR.01 MIME type Detection of Model Files

▪ 6.1.2 FR.02 Plausibility Check (Extension vs. Content)

▪ 6.1.3 FR.03 Readable Error Message After Plausibility Check

▪ 6.1.4 FR.04 Table of Contents in XML Visualization

▪ 6.1.5 FR.05 AAS Generator Wizard and Property Adoption

▪ 6.1.6 FR.06 Background Processing for KBL/VEC to AAS Submodels

▪ 6.1.7 FR.07 REST API for Targeted Data Retrieval

◦ 6.2 Non-functional Requirements

▪ 6.2.1 NFR.01 Usability

▪ 6.2.2 NFR.02 Performance

▪ 6.2.3 NFR.03 Maintainability and Contribution to the Open-Source Project

▪ 6.2.4 NFR.04 Documentation

▪ 6.2.5 NFR.05 Compatibility

▪ 6.2.6 NFR.06 Error Handling

▪ 6.2.7 NFR.07 Availability (Demo)
7. Links to submitted BaSyx Feature Issues and Pullrequest

1. Glossary

Term	Definition
AAS	Asset Administration Shell – standardized digital representation of an asset.
Submodel	Structured data container inside an AAS.
KBL	XML-based format for cable harness data exchange.
VEC	Vehicle Electric Container – XML standard for electric/electronic system data.
MimeType	Internet media type describing file content type (e.g., <code>application/xml</code>).
DOM	Document Object Model – tree representation of XML.
REST API	HTTP-based programming interface used for structured data access.
BaSyx	Eclipse open-source framework for managing and hosting AAS.
Node ID	Identifier for an XML element or AAS element used for referencing.

2. Scope

This Software Requirements Specification (SRS) defines the Use-Cases for extending the Eclipse BaSyx user interface, including the editor and viewer plugins as well as the related REST API backend. In addition functional and non-functional requirements will be listed. It establishes a common understanding among stakeholders and provides the basis for development, testing, and validation of the software solution.

The goal of the system is to enable seamless and automated integration of engineering data from external sources (e.g., KBL or VEC files) into the Asset Administration Shell (AAS). The specification focuses on translating user and business needs into clear, implementable software requirements.

3. Introduction

The project aims to extend Eclipse BaSyx to support direct import, validation, and integration of external model files into the AAS. The system will automate data plausibility checks, linking, and extraction of relevant technical information into standardized submodels. This reduces manual work and improves data quality and consistency.

Additionally, the REST API will be enhanced to provide access to data points within XML-based files, allowing this information to be visualized in a structured and user-friendly manner in the viewer. The resulting system will deliver an efficient and intelligent interface for managing and understanding complex engineering data within the AAS environment.

4 Component Overview

Frontend (BaSyx UI):

- File Upload Component
- MIME Type Detection & Plausibility Check (Updated)
- XML Viewer (Tree-Based Navigation) (New)
- AAS Generator Wizard (New)
- Error Message Handling System

Backend (AAS Server Extensions):

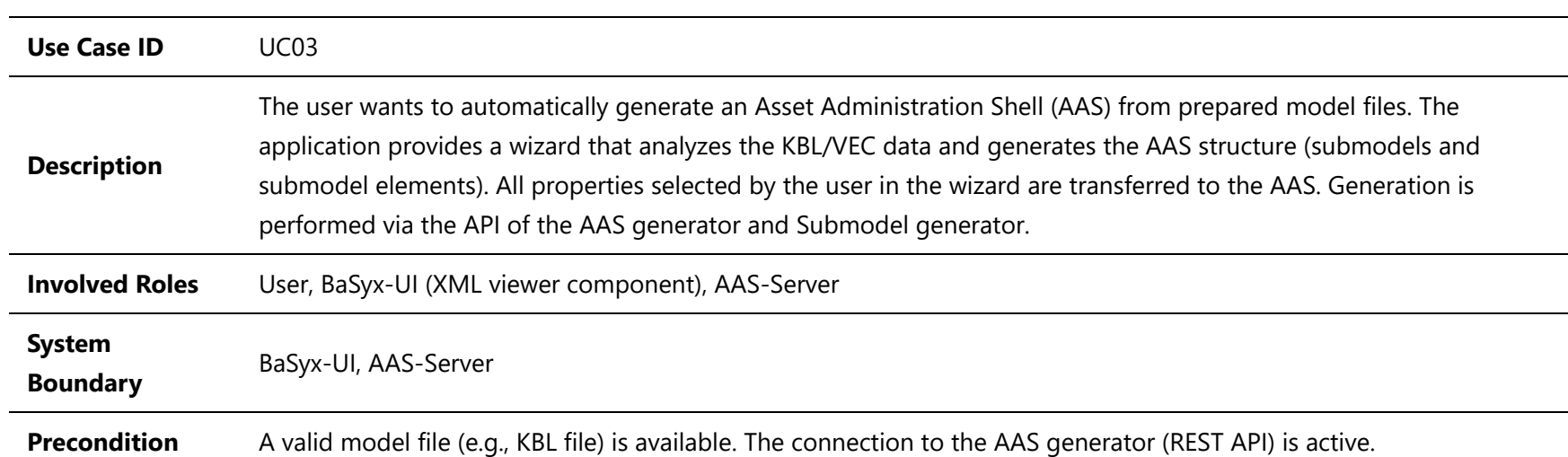
- KBL/VEC Data Extraction Service (New)
- XML Query REST API Endpoint (New)

5. Use Cases

5.1 UC01 Import with plausibility check and MIME type detection

Use Case ID	UC01
Description	The user wants to import an external model file with a specific format (KBL, VEC) or a general data format into the application. The application performs a plausibility check to ensure that the file extension and the actual content structure of the file match. After successful verification, the correct MIME type is set and the file is made available for further processing.
Involved Roles	User, BaSyx-UI (Editor-Plugin), AAS-Server
System Boundary	BaSyx-UI, AAS-Server
Precondition	The user is in the BaSyx UI editor and has an AAS open for editing. The model file exists on the user's local system.
Postcondition on Success	The file has been successfully validated. The correct MIME type has been determined and stored. The file is now available for subsequent processing steps (e.g., AAS generation).
Triggering Event	The user selects the file using an import function and starts the upload process.

Flowchart UC01

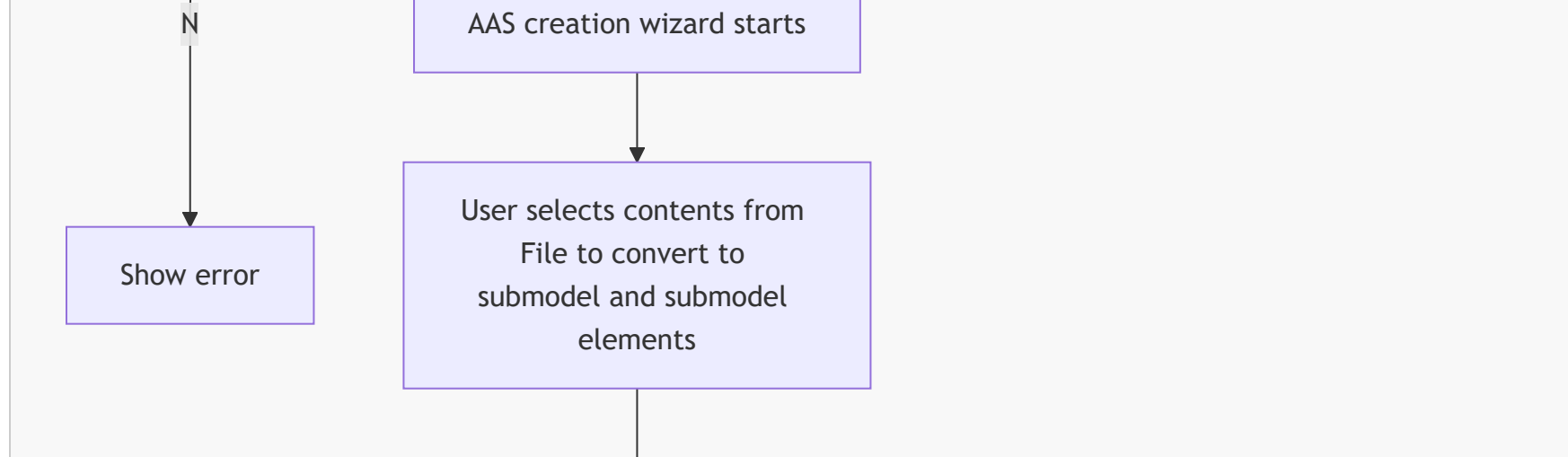


SRS-UC01-Graph

5.2 UC02 XML viewer with navigation and display functions

Use Case ID	UC02
Description	The user wants to view the contents of an XML file (e.g., imported KBL/VEC or AML data). The application provides an XML viewer that displays a table of contents for the XML document to facilitate navigation. The user can navigate to the appropriate section of the document using the table of contents.
Involved Roles	User, BaSyx-UI (XML viewer component)
System Boundary	BaSyx-UI, AAS-Server
Precondition	A valid XML file has already been successfully imported.
Postcondition on Success	The XML content is displayed clearly. Users can navigate to the relevant section of the XML file using the table of contents and view the details (including IDs) of the individual nodes.
Triggering Event	The user opens the viewer of an asset.

Flowchart UC02

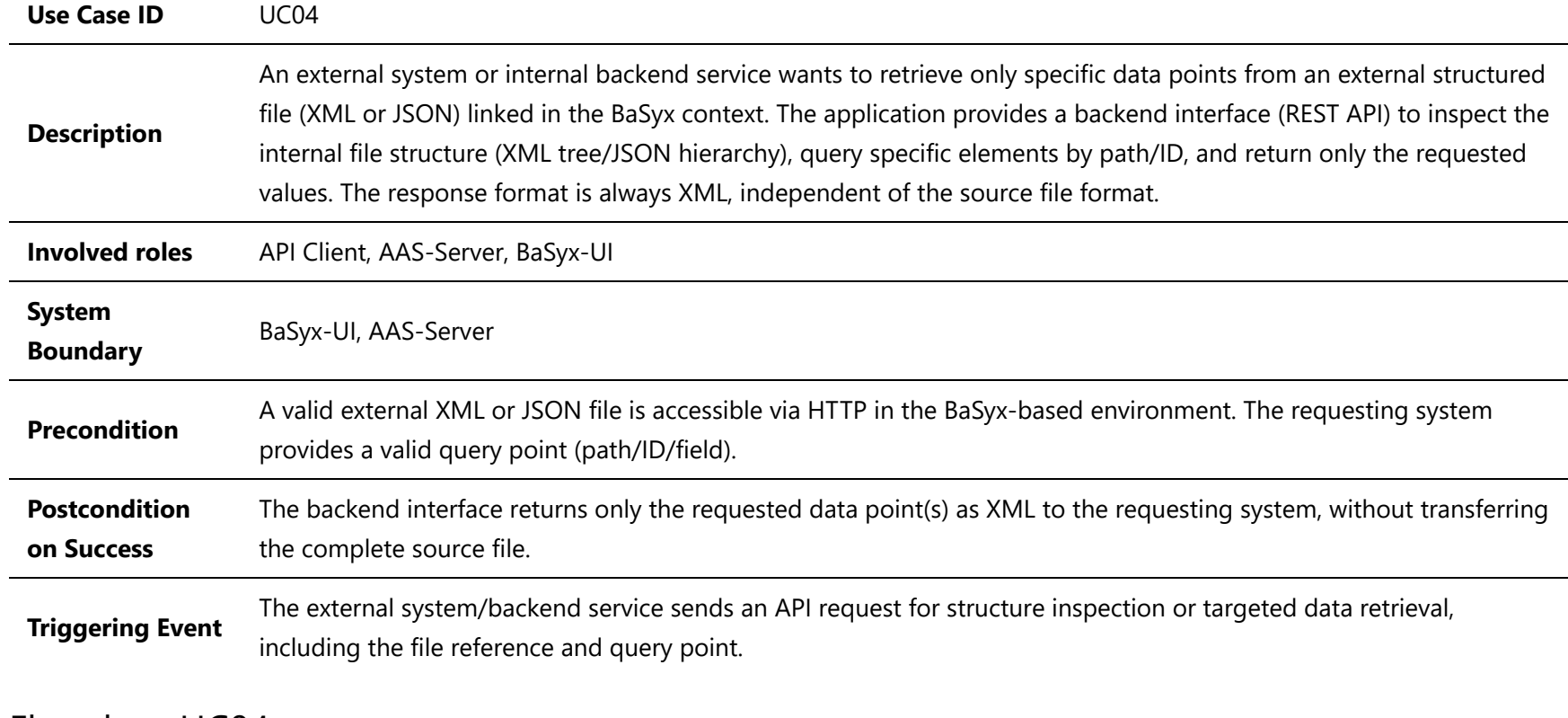


SRS-UC02-Graph

5.3 UC03 AAS generator from KBL/VEC

Use Case ID	UC03
Description	The user wants to automatically generate an Asset Administration Shell (AAS) from prepared model files. The application provides a wizard that analyzes the KBL/VEC data and generates the AAS structure (submodels and submodel elements). All properties selected by the user in the wizard are transferred to the AAS. Generation is performed via the API of the AAS generator and Submodel generator.
Involved Roles	User, BaSyx-UI (XML viewer component), AAS-Server
System Boundary	BaSyx-UI, AAS-Server
Precondition	A valid model file (e.g., KBL file) is available. The connection to the AAS generator (REST API) is active.
Postcondition on Success	A new AAS has been successfully generated, in which all supported data selected by the user is mapped from the model file as submodel elements. The created AAS is stored on the AAS-Server. Two submodels were created, "HandoverDocumentation" with the original File attached and "TechnicalData" with the extracted datapoints.
Triggering Event	The user selects or expands the linked XML file element in the viewer.

Flowchart UC03



SRS-UC03-Graph

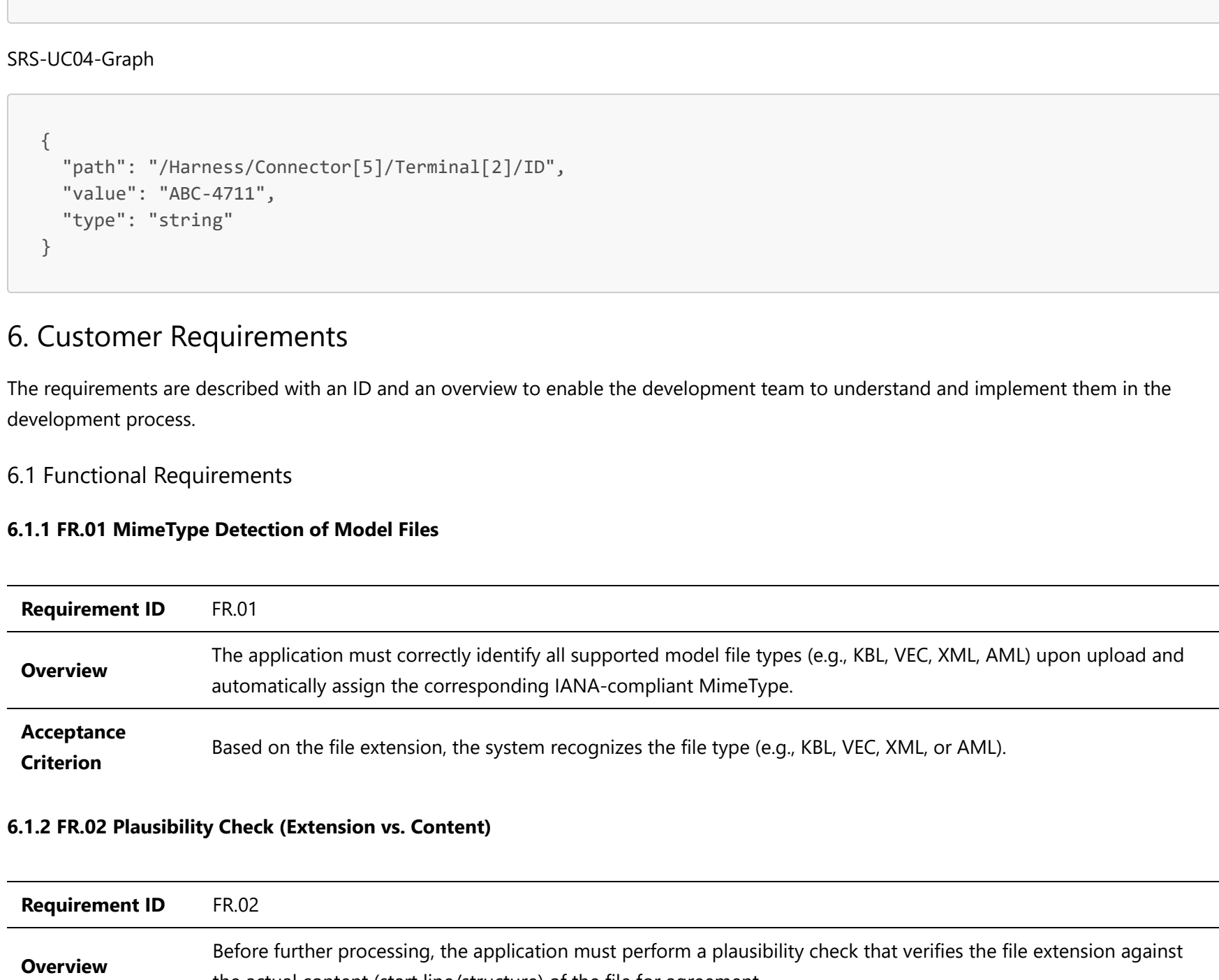
Example AAS Structure After Generation



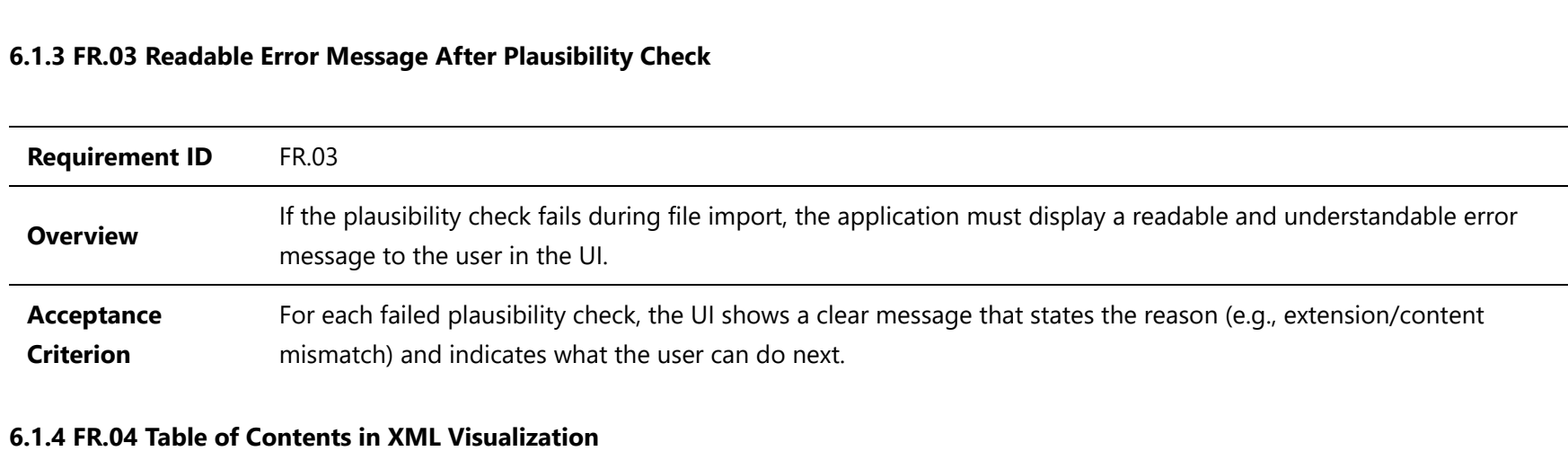
5.4 UC04 Automated extraction of specific XML entries from the AAS

Use Case ID	UC04
Description	An external system or internal backend service wants to retrieve only specific data points from an external structured file (XML or JSON) linked in the BaSyx context. The application provides a backend interface (REST API) to inspect the internal file structure (XML tree/JSON hierarchy), query specific elements by path/ID, and return only the requested values. The response format is always XML, independent of the source file format.
Involved roles	API Client, AAS-Server, BaSyx-UI
System Boundary	BaSyx-UI, AAS-Server
Precondition	A valid external XML or JSON file is accessible via HTTP in the BaSyx-based environment. The requesting system provides a valid query point (path/ID/field).
Postcondition on Success	The backend interface returns only the requested data point(s) as XML to the requesting system, without transferring the complete source file.
Triggering Event	The external system/backend service sends an API request for structure inspection or targeted data retrieval, including the file reference and query point.

Flowchart UC04



SRS-UC04-Graph



6. Customer Requirements

The requirements are described with an ID and an overview to enable the development team to understand and implement them in the development process.

6.1 Functional Requirements

6.1.1 FR.01 MIME type Detection of Model Files

Requirement ID	FR.01
Overview	The application must correctly identify all supported model file types (e.g., KBL, VEC, XML, AML) upon upload and automatically assign the corresponding IANA-compliant MIME type.
Acceptance Criterion	Based on the file extension, the system recognizes the file type (e.g., KBL, VEC, XML, or AML).

6.1.2 FR.02 Plausibility Check (Extension vs. Content)

Requirement ID	FR.02
Overview	Before further processing, the application must perform a plausibility check that verifies the file extension against the actual content (start line/structure) of the file for agreement.
Acceptance Criterion	In case of a conflict between extension and content (e.g., an .xml file starting with PK), the upload is aborted, and an informative error message (see FR.03) is issued.

6.1.3 FR.03 Readable Error Message After Plausibility Check

Requirement ID	FR.03
Overview	If the plausibility check fails during file import, the application must display a readable and understandable error message to the user in the UI.
Acceptance Criterion	For each failed plausibility check, the UI shows a clear message that states the reason (e.g., extension/content mismatch) and indicates what the user can do next.

6.1.4 FR.04 Table of Contents in XML Visualization

Requirement ID	FR.04
Overview	The Viewer plugin must display the data of an attached XML file in a structured table of contents to enable navigation through the document.
Acceptance Criterion	When a user opens an AAS and clicks on the Viewer, a new view displays the table of contents and the XML file.

6.1.5 FR.05 AAS Generator Wizard and Property Adoption

Requirement ID	FR.05
Overview	The application must provide a Wizard that uses the data extracted from the KBL/VEC files for the automatic generation of the AAS model and the associated Submodel Elements. All properties selected by the user must be adopted.
Acceptance Criterion	The Wizard guides the user through the generation process. Upon completion, the generated AAS model contains all selected KBL/VEC data as Submodel Elements.

6.1.6 FR.06 Background Processing for KBL/VEC to AAS Submodels

Requirement ID	FR.006
Overview	As background processing of FR.05, the client application must automatically parse KBL/VEC files and transform the extracted data into the correct AAS submodels and submodel elements according to the defined mapping rules. Only the validated and correctly generated submodels are sent to the backend.
Acceptance Criterion	For a valid KBL/VEC input, client-side processing generates the expected submodels and correctly assigned submodel elements with accurate values and structure. The backend receives only valid submodels; invalid or incomplete mappings are not transmitted.

6.1.7 FR.07 REST API for Targeted Data Retrieval

Field	Content
Requirement ID	FR.07
Overview	The backend interface (REST API) must be extended to allow external systems to inspect and query external structured files (XML/JSON) in a targeted way. The API must support structure inspection (XML tree/JSON hierarchy) and selective extraction of specific entities by path/ID/field. The response payload must always be returned as XML. (Reference to UC04).
Acceptance Criterion	Endpoints for structure inspection and targeted querying exist. For a valid request, the API returns only the requested element(s)/value(s) as XML and does not return the full source document. For invalid query paths, the API returns a clear client error (e.g., HTTP 400/404).

6.2 Non-functional Requirements

6.2.1 NFR.01 Usability

Requirement ID	NFR.001
Overview	The new functionalities in the editor and viewer must be intuitive and well integrated into the existing workflow of the BaSyx UI.
Acceptance Criterion	A user who is familiar with the BaSyx UI can perform the main use cases (uploading a file, viewing XML content) without consulting the documentation. The workflow for each use case is clear and logical.

6.2.2 NFR.02 Performance

Requirement ID	NFR.002
Overview	The parsing of the files and the data extraction should be efficient to ensure a responsive user experience.
Acceptance Criterion	The server-side processing (parsing, extraction, populating the submodel) of a standard-sized model file (up to 5 MB) must be completed in under 5 seconds. The UI provides feedback (e.g., a loading indicator) during this process.

6.2.3 NFR.03 Maintainability and Contribution to the Open-Source Project

Requirement ID	NFR.003
Overview	The developed code must comply with the coding standards of the Eclipse BaSyx project and be well documented to facilitate maintenance and future contributions from the open source community.
Acceptance Criterion	The code follows existing project conventions. All new public classes and methods are documented. The implementation is covered by unit tests. A pull request for the new features is created and submitted to the official BaSyx repositories.

6.2.4 NFR.04 Documentation

Requirement ID	NFR.004
Overview	Clear and structured user documentation for the new functionalities must be created.
Acceptance Criterion	An online user documentation is created. It explains the new functions with step-by-step instructions and screenshots. The tutorials for setting up the BaSyx infrastructure are evaluated and improved where gaps have been identified.

6.2.5 NFR.05 Compatibility

Requirement ID	NFR.005
Overview	The extensions must be fully compatible with the target version of the BaSyx UI and the backend infrastructure and must not negatively affect existing functionalities.

[Acceptance Criterion] All existing, unchanged functionalities of the BaSyx UI continue to function as expected after the integration of the new features. The solution can be built and run within the standard BaSyx build chain.]

6.2.6 NFR.06 Error Handling

Field	Content
Requirement ID	NFR.06
Overview	The system must correctly handle errors (e.g., corrupt files, API failures, invalid query paths) and return clear, informative error messages (UI) or correct HTTP status codes (API).
Fit Criterion	no crash occurs upon generation error (UC03); a 404 Not Found or 40 Bad Request is returned upon invalid API path (UC04).

Figure 4: File Error Sketch

6.2.7 NFR.07 Availability (Demo)

Requirement ID	NFR.07
Overview	The final functionalities must be hosted on a publicly accessible demo server and be executable.
Fit Criterion	The Use Cases implemented in the project can be tested live by project stakeholders via a provided URL without requiring a VPN or special software installations.

7. Links to submitted BaSyx Feature Issues and Pullrequest

The corresponding Issues we opened on the [BaSyx-Github Issues page](#) are:

- [FEATURE] Import AAS from .KBL and .VEC files #1201
 - [FEATURE] XML Tree View with Distinguishable Node Labels and Navigation #1200
 - [FEATURE] REST API for Selective Access to Content within External XML/JSON Files #1180
 - [FEATURE] File Validation for XML, KBL, VEC #1179
- The Pullrequest submitting the changes according to those Feature-Issues can be found [here](#).